

HTTP Clients

Prof. Dr. Peter Braun

9. November 2024

1 Titlepage

HTTP Clients

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Goals of this Unit

Learning Goals

- You learn how to implement an HTTP client in Java
- You know different libraries for HTTP clients
- You implement your first HTTP client

Welcome to this unit about HTTP clients in Java. In this unit, we will look at implementing HTTP communication on the client side. We assume that you understand the HTTP protocol, know what HTTP verbs are, and see the structure of HTTP messages. All this was covered in the last unit. We will show you two ways to implement an HTTP client in Java. The first way uses the standard classes in Java; the other two suggestions use libraries to make programming HTTP communication slightly easier. Using a simple example of a service, we will start with GET requests and then add the other HTTP verbs such as POST, PUT and DELETE requests.

3 Structure of an HTTP Client Request

Structure of an HTTP Client Request

- Open a network connection to a given URL
- Configure the HTTP request message
- Send the request message to the server
- Wait for the response message from the server
- Read the response message and process the result

Let's start by looking at the basic flow of a request to a service. Of course, the client first needs the URL to send the request to. So, the hostname and port are probably other path elements. Then, the client can configure the HTTP request message with additional parameters. For example, he could define in which data format he would like to receive the response message. After sending the message to the service, it waits for the server's incoming response and evaluates it. He checks the status code to see if the server can process the request correctly or if there is an error, and he processes the data in the response message.

4 Implementing Java Clients using HttpURLConnection

Implementing Java Clients using HttpURLConnection

```
1 URL theUrl = new URL(url);
2 HttpURLConnection con = (HttpURLConnection) theUrl.openConnection();
3
4 con.setRequestMethod("GET");
5 con.setRequestProperty("Accept", "application/json");
6
7 int responseCode = con.getResponseCode();
8
9 BufferedReader in = new BufferedReader(
10     new InputStreamReader(con.getInputStream()));
11
12 // reading and processing response
```

- In case of PUT and POST, write data to the output stream of the HttpURLConnection con.

We want to implement a Java client to make such requests. As always, there are numerous approaches to solving this problem. I will present two approaches. Let's start with the simplest implementation, which doesn't require any additional libraries, but all the classes you need are already available in Java. This approach uses the class "HTTP URL Connection". In the first line, the address, initially a string, must be wrapped into an object of type URL. Then, in line 2, you can use the "Open Connection" method to establish a new HTTP connection. In line 4, you can see how the HTTP verb is defined; in line 5, another parameter is set. In case of a GET request, the client should set the parameter to specify the data format by which it would like to receive the response message from the server. We use the JSON format in this case. Later in this unit, we will look at this data format. In line 7, we now access the status code of the response message, and only then do we actually send the request message to the server and wait for the response. In lines 9 to 10, we read the received response message from the connection's input stream.

5 Implementing Java Clients using OkHttp

Implementing Java Clients using OkHttp

```
1 OkHttpClient client = new OkHttpClient();  
2 Request request = new Request.Builder()  
3     .url( url )  
4     .get()  
5     .build();  
6  
7 Response response = client.newCall( request ).execute();
```

- The fluent interface makes the source more readable

The second solution uses a library called OkHTTP. This is a modern and up-to-date approach to working with the HTTP protocol from the client's point of view. The source code is very clear and extremely easy to read. We first need an instance of the `OkHTTP Client` class, which we create in line 1. We then build a request using a so-called builder pattern, with a technique called method chaining at this point. In lines 3-5, you write several method calls in a row and configure such a request with little effort. Finally, in line 7, the configured request is executed. You receive a response that you can process as usual.

6 A Simple Service to Manage Persons

A Simple Service to Manage Persons

- The service manages resources of type `Person`
- `Person` consists of first and last name, date of birth
- Operation `GET` to retrieve all (or selected) objects
- Operation `POST` to create an object
- Operation `PUT` to update an object
- Operation `DELETE` to delete an object

Next, we will start to implement a complete HTTP client. First, we need a server with an appropriate service we want to use via our Java client. For example, we use a straight-forward service for managing “Person” objects. A person is modeled as a data structure consisting of a first name, a last name, and a date of birth. The service allows us to query all persons using the HTTP `GET` operation. Some filters will allow us to search for specific people, for example, by first name. With the HTTP `POST` operation, we create new person objects on the server; with `PUT`, we modify existing records; and with `DELETE`, we delete a record. This service is an extended version of the one you implemented in the last unit. You will find the code for this service in the GitHub repository.

7 Exercise 1: Implement a GET Request

Exercise 1: Implement a GET Request

- Implement a Java class that sends a GET request
- We recommend using the library OkHttp
- URL: `http://localhost:8080/api/persons`
- Read the response body as `String` and print the result to `System.out`

Now that you have learned about the service, here comes your first task. Please implement a first GET request against the given URL to retrieve all person records. Please use the OkHTTP library for this. Read the result from the server as a string and display the text on the screen. Press the button if you want to see the solution.

8 Screencast 1

Screencast 1

The goal was to implement a first HTTP client using the OKHttpClient library since the general structure of configuring the client and configuring a request was shown on the slide before. It was more like replacing some variables here with the correct values. So we have the URL here to the collection resource. We configure the OKHttpClient. We start the GET request here and then we print the status code and we print the raw string that is part of the response body. Let's start the backend and then we can start the client and you can see the result here status code 200 and you can see we have a collection here and the expected first resource.

9 Exercise 2: Deserialize the Response

Exercise 2: Deserialize the Response

- Deserialize a collection of objects of type `Person`
- Print the result to `System.out`

Next, you should extend the solution of the first exercise to process the response message. Apply what you just learned about deserializing JSON into a collection of Java objects of type `Person` in the last unit and process the server's response accordingly. Then, display the persons on the screen. Press the "Continue" button if you want to view the solution.

10 Screencast 2

Screencast 2

We extend the solution for the first exercise now and after we have received the body here, the goal was to deserialize this string into a collection of person objects. Therefore we have to configure the object mapper from Jackson here, because the person class and also the person representation contains the date information. We have to register this module here in order to pass the date format correctly and then we have to apply this kind of trick here to inform Jackson about the correct data types. We want to have a list of person here. You learned about this way to provide this generic data type information in the last unit already. We have to create an object here of type reference and by using this object the Java bytecode carries enough information about the generic type here so that Jackson can create actually a list and put some person objects into this list. Finally we just print here the result. We start the backend again and we start this program. Now you can see the brackets also indicate in Java to string a collection. In the collection we have one person object and the way to print or to format here one person object comes from the class person because here we have a toString method that exactly defines the format that should be used here.

11 Example of a POST Request with OkHttp

Example of a POST Request with OkHttp

```
1 OkHttpClient client = new OkHttpClient();
2
3 RequestBody body = RequestBody.create(
4     MediaType.parse("application/json"), json );
5
6 Request request = new Request.Builder()
7     .post( body )
8     .url( url )
9     .build();
10
11 Response response = client.newCall( request ).execute();
```

- The String named `json` in line 4 contains the JSON representation of a Java object of type `Person`

Next, let's look at a POST request. In line 1, we first create a client of type `OkHttpClient`. In line 3, we now create the content area for the POST request. We have to set the media type for the content. In principle, we want to send JSON, but we don't care how we created the JSON content. The data is already as variable `"json"` in line 3. Then, starting on line 6, we make the POST Request, and in line 7, we specify what content we want to send along. Then, in line 11, we send the request to the server and await the response.

12 Exercise 3: Send a POST Request

Exercise 3: Send a POST Request

- Serialize an object of type `Person`
- Send the POST request
- Check the status code in the response message
- Read the `Location` response header
- Send a GET request to this URL

In the last exercise in this unit, you should extend the implementation of a POST request and serialize an object of type “Person” in the variable “json” you saw on the last slide. In addition, please process the response sent by the server. The status code should be 201 for “Created”. This means the server informs the client that the new resource was created and stored in the database. The header of the response message contains a field named “Location,” which includes the full URL of the new resource. Read this URL from the header and store it in some variable. Finally, send a GET request to this URL. You should receive the JSON representation of the new resource.

13 Screencast 3

Screencast 3

So also for this exercise the general outline or structure of how to implement a post request was shown on the slide before. But we have to give some parameters here. So we have the URL, we create this JSON representation as a string of a person. Then we configure the OK HTTP client and we send a post request. This is defined here. And after we have received the status code here, the goal was to read the location header from the response header and to start a new GET request, which is configured here to this location header and then send the GET request to the backend. And we just print the status code here and this should be 200. We start the backend. OK, this looks good. So first after the post request we have this 201 status code and then status code 200 for the GET request.

14 Summary

Summary

- You have learned about HTTP libraries
- You have implemented your first HTTP client

That's it for this unit. That was a quick one. You should now understand how to send HTTP requests on the client side. In this course, we do not deal with frontend programming, but we must write some software for the client side to demonstrate how clients can be implemented, and most importantly, we have to test the server.

15 Literature

Literature

- Amundsen, Michael. RESTful Web Clients: Enabling Reuse Through Hypermedia. First edition. Beijing: O'Reilly, 2017.
- L. Richardson und M. Amundsen, RESTful Web APIs, First edition, Second release. Beijing Cambridge Farnham Köln Sebastopol Tokyo: O'Reilly, 2015.